

1. 黑盒/白盒对比

黑盒测试	白盒测试
基于规约	基于代码
覆盖声明行为	覆盖实现行为

2. 黑盒测试的特征

原因：

有时无法获取程序代码

可以发现其他测试遗漏的逻辑缺陷（因果关系）

尽早测试，尽早发现功能缺陷

适用于各个测试阶段：单元、集成、系统、回归

黑盒测试：一种基于规格说明，不要求考察代码，以用户视角进行的测试或称：功能测试、基于规格说明的测试

意义（有助于软件产品的总体功能验证）：

检查明确需求和隐含需求

采用有效输入和无效输入

包含用户视角

实施者：专门的软件测试部分；有经验的测试人员

步骤：

- (1) 熟悉规格说明书，理解测试需求

(2) 生成测试用例

(3) 执行测试

(4) 判定测试结果

特点：黑盒测试是一种基于需求的测试

核心目的：确认SRS列出的需求是否都正确实现

前提：

- (1) SRS已经经过仔细评审

(2) 隐含需求已经明确化

优点：

与软件具体实现无关，测试用例在软件变化后仍可用

设计黑盒测试用例可以和软件实现同时进行，压缩项目总开发时间

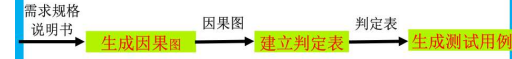
5. 组合测试面临处理大量无效测试用例的事实（多个输入条件关联）

因果分析法原理：

- (1) 软件的输入输出间存在因果逻辑关系，可以用因果图刻画

(2) 因果图可以从SRS中获得

流程：



因果图中的四种关系：

- (1) 恒等：因果共亡亡

(2) 非（~）：因果只存一个

(3) 或（v）：任意一因就有果

(4) 与（^）：两因皆具才有果

因果图的四种输入约束：

- (1) 互斥（E）：至多包含一者

(2) 包含（I）：至少包含一者

(3) 唯一（O）：精确包含一者

(4) 要求（R）：蕴含关系，前件为真后件不假

因果图的一种输出约束：屏蔽：前件为真后件必假

因果图列表：原因的所有组合及相应的结果（某些原因组合可能不存在）

因果图列表表示例									
原因 (可能输入)		1	2	3	4	5	6	7	8
	C1	0	0	0	0	1	1	1	1
	C2	0	0	1	1	0	0	1	1
	C3	0	1	0	1	0	1	0	1
结果 (预期输出)	E1	0	0	0	1	0	1		
	E2	1	1	0	0	0	0		
	E3	1	0	1	0	1	0		

判定表构成：

- (1) 原因：判定表中的条件（测试用例的输入条件）

(2) 结果：判定表中的行动（测试用例的期望输出）

(3) 判定规则：原因与结果的组合（一个测试用例）

判定表示例							
条件 (有效输入)		1	2	3	4	5	6
	C1	0	0	0	0	1	1
	C2	0	0	1	1	0	0
	C3	0	1	0	1	0	1

3. 黑盒测试的实施：

正面测试

负面测试

正面测试：通过正面测试用例产生一组预期输出验证产品需求

负面测试：展示当输入非预期输入时，产品没有失败

对比：

	正面测试	负面测试
测试条件	根据需求说明规格	需求规格说明之外
目的	验证SRS的需求是否得到满足	通过非法输入尝试搞垮软件
覆盖率计算	覆盖需求和条件	无覆盖率
挑战性	根据规格说明设计测试用例	需要高度创造性产出尽可能多的非法输入

用例设计的生成方法：

等价类划分法

因果图/判定表法

边界值分析法

功能分解法

功能分解法：把软件需求的每一个功能加以分解为功能单位，以功能单位为对象进行测试。

4. 等价类：

- (1) 测试相同目标或者暴露相同软件缺陷的一组测试

(2) 输入域的某个子集，该子集中的每个输入数据对揭露软件中的错误是等效的，测试等价类的每个代表值就等价于对这一类其他值的测试

(3) 如果软件行为对一组值来说是相同的，则称这组值为等价类。

等价类划分原理：

- (1) 把程序的输入划分为等价类

(2) 设计一个测试用例来尽可能发现多个错误

划分：产生相同预期输出的一组值叫一个划分

有效等价类：完全满足产品规格说明的输入数据，即有效的、有意义的输入数据构成的集合。可以检验程序是否满足SRS。

无效等价类：不满足程序输入要求或者无效的输入数据构成的集合。可以检验程序能否处理非法输入。

划分准则：

输入数据类型	划分等价类规则	
布尔值	1个有效等价类：TRUE	1个无效等价类：FALSE
取值范围	1个有效等价类：正确取值范围	2个无效等价类：大于和小于取值范围
数据个数	1个有效等价类：正确数据个数	2个无效等价类：大于和小于数据个数
集合	1个有效等价类：正确的集合取值	1个或多个无效等价类
需分别处理的一组输入数据	多个有效等价类：每个输入数据为1个等价类	1个无效等价类
符合某些规则的输入	多个有效等价类：符合某个规则的输入数据为1个等价类	若干个无效等价类

使用等价类生成测试用例：

- (1) 选择划分准则

(2) 根据准则确定有效和无效等价类

(3) 从等价类中选取样本数据覆盖所有等价类（有效要覆盖，无效要单独覆盖）

(4) 根据需求写预期结果

(5) 执行测试

考虑输出条件有助于对输入域进行精细的等价类划分

6. 边界值分析法：对输入和输出的边界进行测试的一种黑盒测试方法。通常作为等价类划分法的补充，此时测试用例来自等价类的边界。

原因：软件的两个主要缺陷源：条件、边界

- (1) 条件：变量取值需要满足的约束

(2) 边界：变量的各种“极限取值”

边界值分析能有效捕获出现在边界处缺陷，利用并扩展了缺陷容易出现在边界处的理念。

使用边界值分析方法设计测试用例：

- (1) 确定边界情况

条件 (有效输入)	C1	0	0	0	0	1	1
	C2	0	0	1	1	0	0
	C3	0	1	0	1	0	1
行动 (预期输出)	A1	0	0	0	1	0	1
	A2	1	1	0	0	0	0
	A3	1	0	1	0	1	0

因果分析法：

- (1) 分析SRS，识别原因和结果
- (2) 在因果图连接原因和结果
- (3) 标明原因之间以及结果之间的约束条件
- (4) 因果图转换为因果图列表进而生成判定表
- (5) 判定表的规则转换为测试用例

判定表的组成：

- (1) 条件桩：列出所有可能问题（条件）
- (2) 条件项：列出条件所有可能取值
- (3) 动作桩：列出可能采取的动作
- (4) 动作项：指出在条件项的各种取值情况下应采取的动作

判定规则：贯穿条件项和动作项的一系列

判定表简化：目标是合并相似规则。在两条或以上规则有相同动作，并且条件项之间存在极大相似，便可以合并。合并后的条件项与取值无关，称“无关条件”

8. 静态白盒测试（结构化分析）：在不执行软件的条件下有条理地仔细审查软件设计、体系结构和代码，从而找出软件缺陷

目的：

- (1) 尽早发现软件缺陷
- (2) 为后继测试中设计测试用例提供思路

优点：

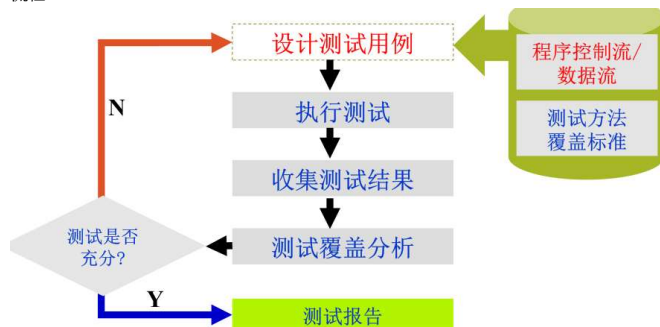
- (1) 可发现某些机器发现不了的错误
- (2) 利用不同人对代码的不同观点
- (3) 对照设计，确保程序能完成预期功能
- (4) 不但能检测出错误，还可以尝试确定错误根源
- (5) 节约计算机资源，增加人工成本
- (6) 尽早发现缺陷，避免后期缺陷修复造成巨大压力

方法：

- 技术评审
- 代码/文档阅读
- 走查
- 专项检查
- 代码、数据、文档审计

9. 动态白盒测试：不但要提供软件源代码，还要提供可执行程序，测试过程需要在计算机上执行程序。

流程：



覆盖标准：对“测试执行到何时才是充分的？”的定量回答

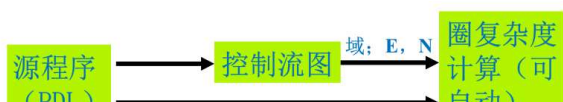
作用：测试软件的一种度量标准，描述程序源代码被测试的程度。一种测试技术通常有一种对应的覆盖标准

方法：

- 基于控制流：
 - 语句测试
 - 条件测试
 - 路径测试
- 基于数据流：数据流测试

11. 基本路径测试：寻找基本路径，根据基本路径构造测试用例，保证每条基本路径至少执行一次。

过程：将程序流程抽象为连通图，通过对图的问题求解得到程序的基本路径



出现在边界处的理念。

使用边界值分析方法设计测试用例：

- (1) 确定边界情况
- (2) 取正好等于，刚刚大于或刚刚小于边界值的值测试，而非典型值或者任意值

测试边界：临近边界的有效数据和刚超过边界的无效数据

测试用例确认方法：

- (1) n：存在边界值的参数个数
- (2) m：边界值条件数

Paul Jorgensen公式：（依次增强）

$4n+1$ ：基本边界测试

$6n+1$ ：健壮性边界测试（也可+x）

$3m$ ：边界条件测试

7. 白盒测试：一种基于源程序或代码的测试方法。一句源程序或代码结构与逻辑生成测试用例，以尽可能多地发现并修改源程序错误。

或称：结构测试 逻辑驱动测试 基于程序的测试

分类：静态、动态

缘由：

- (1) 确保每段代码都被执行，避免缺陷
- (2) 是功能测试的补充
- (3) 能覆盖高层规范说明中忽视的底层细节
- (4) 更易自动化测试

进入条件：编码开始阶段

退出条件：

- (1) 完成测试计划（满足一定覆盖率）
- (2) 发现并修正了错误
- (3) 预算和开发条件

阅读程序代码作为测试工作的一部分得到认同，原因有：

- (1) 软件规模和复杂度
- (2) 开发团队规模
- (3) 开发时限
- (4) 团队的文化和技术背景

10. 基于控制流的方法：逻辑覆盖测试（白盒测试要求对被测程序结构特性做到一定程度的覆盖）

常用覆盖标准		
1	语句覆盖	保证程序中的 每条语句 都执行一遍
2	判定覆盖	保证 每个判断 取True和False至少一次
3	条件覆盖	保证每个判断中的 每个条件 的取值至少满足一次
4	判定条件覆盖	保证 每个条件和由条件组成的判断 的取值...
5	条件组合覆盖	保证 每个条件的取值组合 至少出现一次...
6	路径覆盖	覆盖程序中 所有可能路径

所有覆盖方法都无法绝对保证程序的正确性

语句（所有节点）覆盖：选择足够的测试用例，使得运行这些测试用例时，被测程序的每条可执行语句都至少执行一次

100%的语句覆盖可能很困难：

- 处理错误的代码片段
- 小概率时间（恶作剧），即使是100%的语句覆盖，有时不可达代码

也无法发现某些严重问题！

判定（分支、所有边）覆盖：选择足够的测试用例，使得运行这些测试用例时，被测程序的每个判定的所有可能结果都至少执行一次（即判定的每个分支至少经过一次）

优点：简单，避免了语句覆盖的问题

却带你：忽略了表达式内条件，不能发现每个条件的错误

条件覆盖：选择足够的测试用例，使得运行这些测试用例时，被测程序的每个判定中的每个条件的所有可能结果都至少出现一次。

思想：保证每个判断中的每个条件的取值至少满足一次

不能保证所有分支都被运行

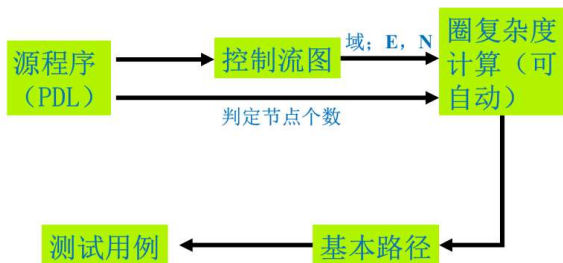
判定条件覆盖：被测程序的每个判定的所有可能结果都至少执行一次，并且每个判定中的每个条件的所有可能结果都至少出现一次（可能无法检测出运算符错误）

思想：保证每个条件和由条件组成的判断的取值取真取假各一次

错误屏蔽：原子条件取值改变不会影响判定结果，因此该条件上的取值错误是不可见的流程：

- (1) 确定哪个测试用例对发现错误是敏感的，哪个组合会产生错误屏蔽

(2) 思考造成错误屏蔽的组合

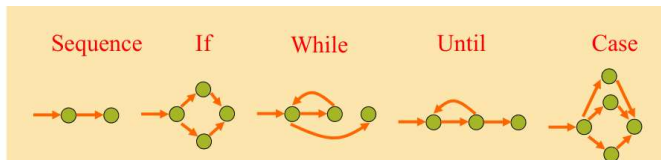


控制流图：用来描述程序中的逻辑控制流

组成：

- (1) 节点：表示一个或多个连续执行的过程语句
- (2) 边：表示控制流
- (3) 域：由边和结点限定的区间

基本结构：



基本路径（独立程序路径）：任何一条贯穿程序的路径，该路径至少包含一条不同于其他路径的边

特性：

- (1) 基本路径集中路径条数唯一，基本路径可以不一样
- (2) 其他路径可以通过基本路径运算得到

度量：圈复杂度

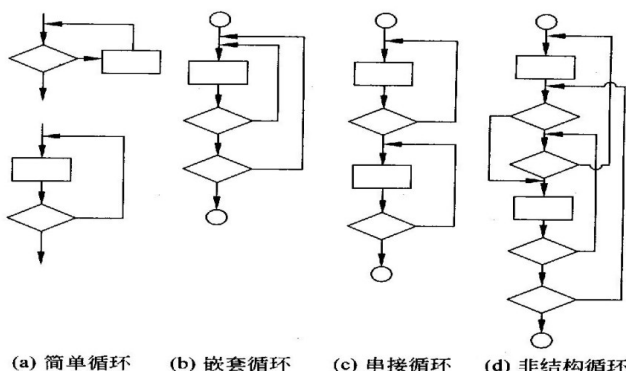
圈复杂度度量基本路径属，是所有语句被执行一次所需测试用例数的上限之一，即执行所有基本路径即可实现100%语句覆盖，但是≠分支覆盖。圈复杂度越高，越有可能蕴含错误。

基本路径集寻找算法：

- (1) 确认从入口到出口的最短路
- (2) 从入口到第一个未被评估真假各一次的条件语句
- (3) 改变该条件语句的判断值
- (4) 按最短路径从这个条件语句到出口
- (5) 重复2-5直至结束

特性：每一条路径都可以表示为边的向量。基本路径是这样一组路径的集合，其他任何路径对应的向量，都可以用这些基本路径对应的向量通过线性运算组合出来

12. 循环处理



简单循环的测试用例构造：

- (1) 跳过整个循环
- (2) 只执行一次循环
- (3) 执行两次循环
- (4) 执行m(m<n)次循环
- (5) 执行n-1, n, n+1次循环

嵌套循环的测试用例构造：

- (1) 先测试最内层循环：所有外层的循环变量取最小值，最内层按简单循环测试
- (2) 由里向外，测试上层循环：此层以外的所有外层循环的循环变量取最小值，此层以内的所有嵌套内层循环的循环变量取“典型值”，该层按简单循环测试
- (3) 重复2，直到各层循环测试完毕
- (4) 对全部各层循环同时取最小循环次数，或者同时取最大循环次数

串接循环的测试用例构造：

模块屏蔽：由于设计取值以及多个因素影响判定结果，因此该设计上的取值由判定不可见的流程：

- (1) 确定哪个测试用例对发现错误是敏感的，哪个组合会产生错误屏蔽
- (2) 不考虑造成错误被屏蔽的组合

条件组合覆盖：

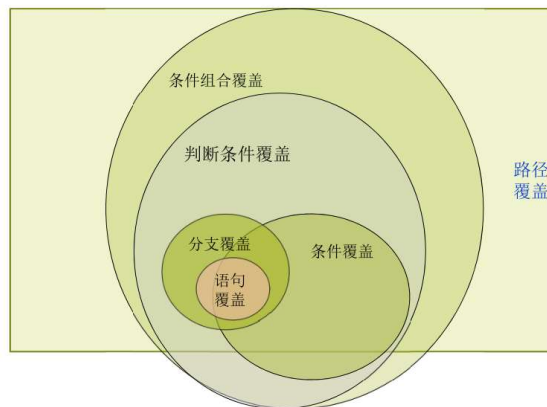
- (1) 选择足够的测试用例，使得每个判定中条件的各种可能组合都至少出现一次
- (2) 所有条件的所有可能取值组合至少出现一次

路径覆盖：选择足够的测试用例，保证每条可能执行到的路径都至少经过一次（如果包含环路，则要求每条环路至少经过一次）

优点：彻底

缺点：路径条数可能以分支的指数级增加、不可达路径存在

路径覆盖考虑了各种判定结果的所有可能组合，但未必能覆盖判定中条件结果的各种可能情况。不能替代条件覆盖和组合覆盖



13. 数据流：变量的定义或使用顺序和变量的可能状态的一种抽象表示

状态：创建（定义）、使用、清除（销毁）

变量存在从创建、使用到销毁的一个完整状态变化过程

数据流测试：一种面向变量定义位置-使用位置覆盖的基于程序结构的测试方法

该方法重点关注变量的定义和使用，在选定的一组代码中搜索某个变量所有的定义位置和使用位置，并检查在程序运行时该变量的值会如何变化，从而分析是否是Bug的产生原因

对比：

数据流测试	路径测试
从数据流角度分析。利用变量关系，通过变量的定义-使用路径设计一系列覆盖标准衡量测试的覆盖率	从控制流角度分析。面向程序的潜在执行路径设计一系列的覆盖标准用语句衡量测试的覆盖率
是路径测试的“真实性”检查，是对基于路径测试的一种改良	

程序：对数据的加工处理过程，对程序的测试可从数据处理流程的角度考虑

数据流图：描述测试对象代码的处理流程，同时也详细描述了代码中变量创建、使用、和销毁的状态。通过检查数据流图来验证测试对象代码中每个变量的状态组合是否正确

Debugging对过程的启发：寻找关于某变量的定义位置和使用位置，思考程序在运行时该变量的值会如何变化，从而分析Bug产生的原因

control flow graph
CFG for control-flow testing

data flow graph
DFG for data-flow testing

注：DFD(data flow diagram)

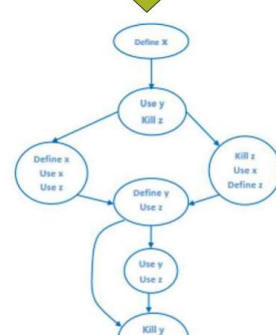
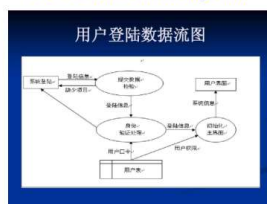


图1 一个数据流图的例

应用：测试技术中，以位置和数据流图为基础的测试技术（如：结构测试、数据流测试、数据流测试）

- (3) 重复2, 直到各层循环测试完毕
 - (4) 对全部各层循环同时取最小循环次数, 或者同时取最大循环次数
- 串接循环的测试用例构造:
- (1) 若串接的各循环互相独立, 分别用简单循环的方式测试
 - (2) 若不独立, 把第一个循环看做外循环, 第二个循环看做内循环, 用测试嵌套循环的方法处理
- 非结构循环: 先结构化再测试。如重复编码法。

作用: 测试变量定义位置和使用位置之间的路径 (定义-使用对/路径)。通过数据流分析而生成的测试集可用来获得针对每个变量的DU路径的100%覆盖。

注意:

- (1) 追踪整个程序代码中每个变量的DU时, 并不需在测试时考虑被测对象的控制流
- (2) 根据程序中某个变量的DU位置来选择程序的测试路径

节点类型:

- (1) 定义节点DEF: 在节点处定义变量
- (2) 使用节点USE: 在节点处使用变量
- (3) 谓词使用P-use: USE是一个谓词
- (4) 计算使用C-use: USE是一个计算
- (5) 输出使用O-use: USE是打印到屏幕
- (6) 定位使用L-use: USE是定位数组位置
- (7) 迭代使用I-use: USE是控制循环次数

路径类型:

- (1) DU路径: 给定路径, 从定义节点 (起始节点) 到使用节点 (终止节点)
- (2) DC路径: 给定DU路径, 除起始节点没有其他定义节点

数据流覆盖测试步骤:

- (1) 对于给定程序, 构造DFG
- (2) 找出所有变量的DU路径
- (3) 考察测试用例对这些路径的覆盖程度, 作为衡量测试效果的度量值

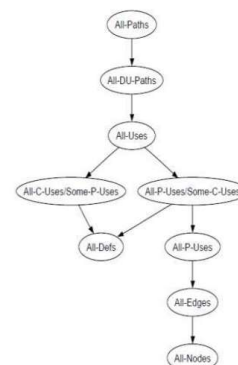
覆盖标准目标:

- (1) 保证所选路径数目总是有限的/可被实际处理的
- (2) 寻找一种系统化的测试路径选择方案, 以帮助发现未知缺陷

Rapps和Weyuker标准:

- (1) All-Paths: 等价于路径覆盖
- (2) All-Edges: 等价于分支覆盖
- (3) All-Nodes: 等价于语句覆盖
- (4) All-Defs: 选择路径中每个变量的每个定义节点都有一条dc-path到该定义的某个使用节点
- (5) All-P-Uses: 选择路径中每个变量的每个定义节点都有一条dc-path到达定一个每个P-use节点
- (6) All-P-Uses/Some C-Uses: 选择路径中每个变量的每个定义节点都有一条dc-path到达该定义的每个P-use节点; 但如果无可达C-use节点, dc-path应到达至少一个C-use节点
- (7) All-C-Uses/Some P-Uses: 选择路径中每个变量的每个定义节点都有一条dc-path到达该定义的每个C-use节点; 但如果无可达C-use节点, dc-path应到达至少一个P-use节点
- (8) All-Uses: 选择路径中每个变量的每个定义节点都有一条dc-path到达该变量的每个使用节点, 若存在定义节点和使用节点间存在多条dc-path, 仅选取其中一条
- (9) All-DU-Paths: 路径中的每个变量的每个定义节点到达该变量的每个使用节点的所有dc-path. 若存在循环, 仅取一个简单循环或者无循环

九个标准的关系(数据流覆盖标准层次结构图):



实际上,这是比那辆的DU路径情况的检查结构。当每个层次结构都检查完成, 则表明针对变量的DU路径达到100%的覆盖。

小结:

- (1) 数据流测试是在程序代码经过的路径上检查变量用法是否正确的一种方法
- (2) 可以发现DU异常 (可能会导致程序失效, 如初始化前读、不使用值)
- (3) 数据流异常不必然导致程序行为失效

优点:

- (1) 揭示隐藏在代码变量DU中的各种错误
- (2) 可以覆盖所有语句、所有分支、所有路径
- (3) 对代码的测试彻底

缺点:

- (1) 无法检测不可达路径
- (2) 不验证需求规格